DELIVERABLE 1 • TECHNICAL REFERENCE MANUAL

# SalesPulse BI: Database Schema Specification

Repository: JRamraju/sales_Dashboard • Engine: MongoDB with Mongoose 9 • Scope: Models, Normalization, Indexing & Lifecycle

## 1.1 Database Architecture & Entity Relationship

SalesPulse BI persists data in **MongoDB** through **Mongoose 9** using two primary collections: `uploadbatches` (model `UploadBatch`) and `salesrecords` (model `SalesRecord`). One upload batch owns many sales records in a strict `1 : N` parent-child relationship linked by `SalesRecord.batchId → UploadBatch._id`.

The schema is intentionally denormalized: there are no separate `customers`, `products`, or `orders` collections. Instead, each validated spreadsheet row is stored as a self-contained order line item, and higher-level entities (distinct orders, customer spend leaderboards, SKU rankings, category/region shares, and monthly revenue series) are derived dynamically at query time via single-collection MongoDB `$facet` aggregation pipelines.

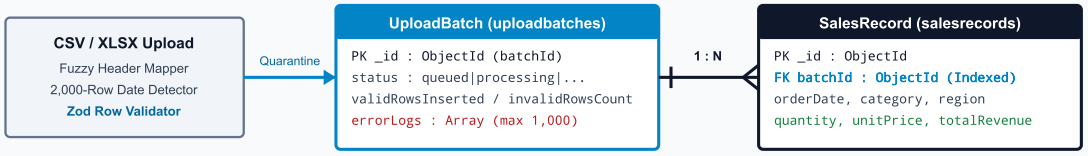



*Figure 1.1: One-to-many relationship between UploadBatch metadata documents and denormalized SalesRecord order lines.*

## 1.2 Sanitization, Fuzzy Header Mapping & Quarantine Rules

- **Tolerant File Formats:** Accepts `.csv` (auto-detecting comma, semicolon, tab, or pipe delimiters via streamed `csv-parser`) and `.xlsx` workbooks (reading the first sheet via `read-excel-file`) up to `MAX_UPLOAD_MB` (default 50 MB). The first row must contain column headers.
- **Fuzzy Header Normalization (`headerMapper.js`):** Column matching ignores case, whitespace, underscores, and punctuation while mapping common aliases to canonical schema fields: `Date → order_date`, `SKU → product_id`, `Qty → quantity`, `Price → unit_price`, `Amount / Total / Revenue → total_amount`, and `Area / Zone / Territory → region`. If two columns map to the same canonical field, the first column wins.
- **Per-File Date-Order Auto-Detection (`dateOrder.js`):** Accepts Excel date cells, Excel serial numbers, ISO-8601 strings, `YYYY-MM-DD`, and `a/b/YYYY` (with `/`, `-`, or `.`). Years must fall within `1900–2100` and represent valid calendar dates (e.g., `31 Feb` is rejected). For ambiguous `a/b/YYYY` dates, the worker inspects the first 2,000 rows: a first number > 12 proves day-first (`dmy`); a second number > 12 proves month-first (`mdy`). If no proof exists, it falls back to `DEFAULT_DATE_ORDER` (default `dmy`) and records an explanatory note on the batch.
- **Currency Cleaning & Row Quarantine (`salesRow.schema.js`):** Monetary cells strip `$`, `₹`, `€`, `£`, commas, and spaces (e.g., `₹1,299.50 → 1299.5`). Completely empty rows are ignored silently. A row is skipped and logged in `errorLogs` (with spreadsheet row number, column name, and reason) if it has a missing `order_id`, `order_date`, or `customer_id`; an unparseable date; a non-integer or `≤ 0` quantity; neither product identifier; neither `unit_price` nor `total_amount`; negative money values; or a discount exceeding the line total.

## 1.3 Input Spreadsheet Columns & Normalization Matrix

Each row in an uploaded CSV or Excel file represents an individual order line item. Table 1.1 defines the expected columns, coercion rules, fallback defaults, and downstream analytical usage.

| Canonical Column | Requirement | Target Type | Normalization & Fallback Rules | Dashboard Analytical Purpose |
|---|---|---|---|---|
| `order_id` | REQUIRED | `String` | Trimmed non-empty string. Multiple rows sharing an `order_id` form lines of a single order. | Counting distinct orders (Total Orders KPI) and grouping multi-item carts. |
| `order_date` | REQUIRED | `Date (UTC)` | Normalized to UTC midnight ( `00:00:00.000Z` ) for date-only values; validated for 1900–2100. | Revenue trend series, date-range filtering, MoM Growth, and Forecasting. |
| `customer_id` | REQUIRED | `String` | Trimmed non-empty string identifying the buyer account. | Counting distinct buyers (Total Customers KPI) and customer spend grouping. |
| `quantity` | REQUIRED | `Number (Int)` | Coerced to integer; must be a positive whole number ( `≥ 1` ). Zero or negative values are quarantined. | Computing units sold per product and line-item extended revenue. |
| `product_id` *or* `product_name` | ONE REQUIRED | `String` | At least one must be present. If one is omitted, it is automatically populated from the other. | Grouping unique SKUs and rendering the Top 10 Products leaderboard. |
| `unit_price` *or* `total_amount` | ONE REQUIRED | `Number (Float)` | Strips currency symbols; rounded to 2 decimals. Missing `total_amount` = `quantity × unit_price − discount` . Missing `unit_price` is derived from total. | Calculating net line revenue ( `totalRevenue` ) and Average Order Value (AOV). |
| `customer_name` | OPTIONAL | `String` | Defaults to `customer_id` when blank or omitted. | Display label in the Top 10 Customers Leaderboard table. |
| `category` | OPTIONAL | `String` | Defaults to `'Uncategorized'` when blank or omitted. | Category multi-select filter and Category Revenue Share Donut Chart. |
| `region` | OPTIONAL | `String` | Defaults to `'Unknown'` when blank or omitted. | Region multi-select filter and Regional Performance Bar Chart. |
| `discount` | OPTIONAL | `Number (Float)` | Absolute monetary amount for the line item (not a percentage). Defaults to `0` ; cannot exceed gross line total. | Subtracted when deriving net revenue from `quantity × unit_price` . |

## 1.4 Batch Lifecycle, Idempotency & Failure Rollback

> **Job State Machine: queued → processing → completed | failed**
>
> When `POST /api/v1/uploads` receives a valid file, it creates an `UploadBatch` in state `queued` . When the BullMQ worker picks up the job, it transitions the batch to `processing` and stamps `startedAt` . Upon finishing all chunks, the batch becomes `completed` (or `failed` on unrecoverable error) and stamps `finishedAt` . Only `completed` batches have immutable records and are eligible for the 120-second in-memory analytics cache.

**Idempotent Reprocessing & Atomic Cleanup:** Before inserting any chunks, the worker executes `SalesRecord.deleteMany({ batchId })` to clear any leftover records from a prior stalled attempt. If an unhandled error occurs during file streaming or bulk insertion, the worker deletes all `SalesRecord` documents already inserted for that `batchId` and sets `status: 'failed'` with a user-safe `failureReason` , guaranteeing no partially loaded dataset is ever exposed to the dashboard. When a user calls `DELETE /api/v1/uploads/:batchId` , the server

## 2.1 Collection 1: `UploadBatch` (Collection: `uploadbatches`)

Defined in `server/src/models/UploadBatch.js`. Stores one metadata document per uploaded file to track background worker progress, date-parsing notes, and quarantined row diagnostics. Its `_id` serves as the `batchId` across all REST and WebSocket payloads.

| Field Name | BSON / Mongoose Type | Default / Constraints | Description & Runtime Behavior |
|---|---|---|---|
| `_id` | `ObjectId` | Auto-generated PK | Unique identifier aliased as `batchId` throughout the API. |
| `fileName` | `String` | `required: true`, `trim: true` | Original user-uploaded filename (e.g., `MOCK_DATA(1).csv`). |
| `status` | `String` (Enum) | `'queued'` (Indexed) | One of `queued`, `processing`, `completed`, or `failed`. |
| `totalRowsRead` | `Number` | `default: 0` | Total non-empty data rows read from the spreadsheet (aliased as `totalRows` in list view). |
| `validRowsInserted` | `Number` | `default: 0` | Number of validated rows successfully bulk-inserted into `salesrecords`. |
| `invalidRowsCount` | `Number` | `default: 0` | True count of quarantined rows (preserved accurately even when `errorLogs` is capped). |
| `processedRows` | `Number` | `default: 0` | Live counter updated after each 1,000-row chunk (persisted to MongoDB at most once/sec). |
| `progress` | `Number` | `default: 0` (`0–100`) | Live completion percentage; for streamed CSVs, estimated from bytes read until completion. |
| `errorLogs` | `Array<{rowNumber, reason, field?}>` | `default: []`, `_id: false` | Quarantine details where `rowNumber` is the 1-based spreadsheet row (header = row 1). Capped at `MAX_ERROR_LOGS` (default 1,000) to stay far below MongoDB's 16 MB BSON limit. |
| `dateOrder` | `String` | `'dmy'` or `'mdy'` | Records whether ambiguous `a/b/YYYY` dates in this file were parsed day-first or month-first. |
| `notes` | `Array<String>` | `default: []` | Human-readable audit remarks explaining how date order was detected or defaulted. |
| `failureReason` | `String` | Optional (set when `failed`) | Sanitized, user-safe explanation of why ingestion failed (e.g., missing required headers). |
| `uploadedAt` | `Date` | `default: Date.now` (Indexed) | Creation timestamp used by `GET /uploads` to sort batches newest-first (limit 200). |
| `startedAt` | `Date` | Optional | Timestamp recorded when the BullMQ worker begins processing the job. |
| `finishedAt` | `Date` | Optional | Timestamp recorded when the job reaches `completed` or `failed`. |

## 2.2 Representative `UploadBatch` Document

```
{
  "_id": "6720f1c2a1b2c3d4e5f60718",
```

## 3.1 Collection 2: `SalesRecord` (Collection: `salesrecords`)

Defined in `server/src/models/SalesRecord.js`. Stores one document per validated order line item. Inserted in unordered chunks of 1,000 via `insertMany(chunk, { ordered: false, lean: true })` to maximize write throughput.

| Field Name | BSON / Mongoose Type | Validation / Rules | Description & Analytical Role |
|---|---|---|---|
| `_id` | `ObjectId` | Auto-generated PK | Unique document identifier for the line item. |
| `batchId` | `ObjectId` | `required`, `ref: 'UploadBatch'` | Owning upload batch ID; leading key of the compound index. |
| `orderId` | `String` | `required: true` | Order identifier; grouped via `$addToSet` / `$group` to count distinct orders. |
| `orderDate` | `Date` | `required: true` (UTC) | Transaction timestamp in UTC; second key in the compound index for range scans. |
| `customerId` | `String` | `required: true` | Customer identifier; grouped to count distinct buyers and customer lifetime spend. |
| `customerName` | `String` | `required: true` | Customer display name (falls back to `customerId` if absent in source file). |
| `productId` | `String` | `required: true` | Product SKU identifier (falls back to `productName` if absent in source file). |
| `productName` | `String` | `required: true` | Product display title (falls back to `productId` if absent in source file). |
| `category` | `String` | `required: true` | Product category (defaults to `'Uncategorized'`); third key in the compound index. |
| `region` | `String` | `required: true` | Sales region/territory (defaults to `'Unknown'`); fourth key in the compound index. |
| `quantity` | `Number` | `required: true` (Positive Int) | Units sold on this order line; summed for product unit volume metrics. |
| `unitPrice` | `Number` | `required: true` (2 decimals) | Per-unit price rounded to 2 decimal places (derived from total when omitted). |
| `discount` | `Number` | `default: 0` (2 decimals) | Absolute line-item discount subtracted from gross price. |
| `totalRevenue` | `Number` | `required: true` (2 decimals) | Net line revenue: uses supplied `total_amount` or `quantity × unitPrice − discount`. |

## 3.2 Compound Indexing Strategy & Index Hygiene

> **Single Compound Index: { batchId: 1, orderDate: 1, category: 1, region: 1 }**
>
> Every analytics query in SalesPulse BI is scoped to a mandatory `batchId` and optionally narrows by `orderDate` range, `category` list, and `region` list. This single compound index satisfies batch scans, date-bounded time-series queries, and multi-select filter combinations via index prefix matching. Earlier iterations also defined single-field indexes on `orderId`, `orderDate`, `category`, and `region`; because no query filters on those fields without `batchId`, they were never selected by the query planner and slowed down bulk inserts by 30–40%. They were removed—set `SYNC_INDEXES=true` for one startup on an existing database to drop obsolete indexes automatically.

## 3.3 Representative `SalesRecord` Document